

Reviewer Pack — Minimal Rank of Quandle Operations Bounds Communication Comp...

Entry 9950d56d296d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 08:14:09 UTC

Minimal Rank of Quandle Operations Bounds Communication Complexity for Symmetry Detection

Entry ID: 9950d56d296d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 08:14:09 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Communication Complexity

Statement:

{‘one’: ‘For any symmetric boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal rank of the quandle operations on the syntactic monoid of the negation of f is $\Theta(n/\log n)$.’, ‘two’: ‘This rank bounds the communication complexity for determining whether a given function f is symmetric.’, ‘three’: ‘For any symmetric function with n variables, there exists a quandle operation of minimal rank that corresponds to its symmetry.’}

Rationale (proposer’s reasoning):

{‘one’: ‘Quandles provide an algebraic structure that captures the symmetries in a set of transformations, which may reveal new insights into the complexity of communication problems.’, ‘two’: ‘The minimal rank of quandle operations could potentially serve as a novel measure for the symmetry of boolean functions, linking algebraic structures with complexity theory.’, ‘three’: ‘This bridge might expose hidden structure related to symmetry detection and could lead to more efficient algorithms.’}

Taxonomy category: Quandle_Theory × Communication_Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2d1848ed4ffff65a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture suggests that for each generated symmetric boolean function f , if the minimal rank of quandle operations on the syntactic monoid of the negation of f is within $\Theta(n/\log n)$, then it supports the conjecture.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	1.00	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Rank Quandle Operations AND Communication Complexity - Symmetry Detection Communication Complexity IN Quandle Theory - Quandle Operation Minimal Rank Bound Communication Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_symmetric_function(n):
    f = [random.randint(0, 1) for _ in range(2**n)]
    for i in range(2**n):
        x = i
        while x != 0:
            if f[x] != f[i ^ ((1 << (n - 1)) - 1)]:
                return None
            x = x & (x - 1)
    return f

def is_symmetric(f, n):
    for i in range(2**n):
        if f[i] != f[i ^ ((1 << (n - 1)) - 1)]:
            return False
    return True

def compute_syntactic_monoid(f, n):
    monoid = set()
    for x in range(2**n):
        for y in range(2**n):
            if f[x] == f[y]:
                monoid.add((x, y))
    return monoid

def apply_quandle_operation(monoid, operation):
    new_monoid = set()
```

```

    for (x, y) in monoid:
        new_x = operation(x)
        new_y = operation(y)
        if (new_x, new_y) not in new_monoid:
            new_monoid.add((new_x, new_y))
    return new_monoid

def compute_quandle_rank(monoid):
    n = len(monoid)
    if n == 0:
        return 0
    adjacency_matrix = [[0] * n for _ in range(n)]
    for i, (x1, y1) in enumerate(monoid):
        for j, (x2, y2) in enumerate(monoid):
            if x1 == x2 and y1 == y2:
                adjacency_matrix[i][j] = 1
    rank = 0
    for _ in range(n):
        found = False
        for i in range(n):
            if sum(adjacency_matrix[i]) > 0:
                rank += 1
                for j in range(n):
                    if adjacency_matrix[i][j]:
                        for k in range(n):
                            if adjacency_matrix[j][k]:
                                adjacency_matrix[i][k] = 1
                        found = True
                        break
            if not found:
                break
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40]
    results = []
    for n in n_values:
        f = generate_random_symmetric_function(n)
        if f is None:
            continue
        if not is_symmetric(f, n):
            continue
        monoid = compute_syntactic_monoid(f, n)
        rank = compute_quandle_rank(monoid)
        results.append(rank)
    if len(results) == 0:
        return {
            "metric_name": "quandle_rank",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }
    mean_rank = sum(results) / len(results)
    expected_rank = n_values[-1] / math.log(n_values[-1])

```

```

return {
    "metric_name": "quandle_rank",
    "metric_value": mean_rank,
    "instances_tested": len(results),
    "conjecture_holds": abs(mean_rank - expected_rank) <= 0.5 * expected_rank,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 3 for i in range(5, 8)]
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])
    mean_rank = sum(results) / len(results)
    support_fraction = sum(1 for r in results if abs(r - n_values[-1] / math.log(n_values[-1])) <= 24)
    if support_fraction >= 24:
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction/len(results)}")
    elif any(not r for r in results):
        first_failing_seed = seeds[results.index(None)]
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print(f"RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it was unable to verify the conjecture's conditions. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within the given time frame.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11685
2	preregistration	ollama_remote	glm4:latest	0	0	6413
3	novelty	ollama_remote	glm4:latest	0	0	4437
4	novelty	ollama_remote	glm4:latest	0	0	8943
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36454
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14169
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13522
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12699
9	judge	ollama_remote	glm4:latest	0	0	15178

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 123501 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9950d56d296d.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9950d56d296d.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9950d56d296d.tar.gz` (if generated)